PREZENTAREA 2 DIN 6 · CURS 3–4 · LABORATOR 4–5

Funcții de ordin înalt

Lambda, currying, compunere, map, filter și fold

ghci

```
ghci> foldr (+) 0 . map (^2) . filter (>0) $
[1,-2,3]
10
```

Ce facem azi

00
Recapitulare
tiparul pe care l-am tot scris recursiv
Prezentarea 1

01
Funcții ca valori
lambda, currying, secțiuni
Curs 3 · Lab 4

02
Compunere
operatorii . și $
Curs 3

03
map și filter
transformă și selectează
Curs 3 · Lab 4

04
fold
foldr, foldl: agregare
Curs 3–4 · Lab 5

05
Lazy, din nou
map, filter, fold pe liste infinite
Curs 4

06
Exerciții
încălzire + opt probleme cu soluții
Lab 4 · Lab 5

07
Recapitulare
capcane, fițuică, quiz

Ce tot scriam în prezentarea 1

transformă fiecare

```
squares :: [Int] -> [Int]
squares []     = []
squares (x:xs) =
  x * x : squares xs
```

Tiparul are un nume: map

păstrează unele

```
positives :: [Int] -> [Int]
positives [] = []
positives (x:xs)
  | x > 0 = x : positives xs
  | otherwise = positives xs
```

Tiparul are un nume: filter

adună tot

```
sum :: [Int] -> Int
sum []     = 0
sum (x:xs) = x + sum xs
```

Tiparul are un nume: fold



Ce diferă între ele? Doar operația aplicată elementului și condiția. Restul e același schelet. Azi transformăm scheletul în funcții care primesc diferența ca argument.

01

Funcțiile sunt valori

Funcții anonime (lambda), currying, aplicare parțială și secțiuni (Curs 3, Laborator 4).



Lambda: funcții fără nume



din Curs 3

```
ghci> (\x -> x + 1) 3
4
ghci> inc = \x -> x + 1
ghci> add = \x y -> x + y
ghci> add 2 5
7
ghci> map (\x -> x + 1) [1,2,3,4]
[2,3,4,5]
```

Scrierile `inc x = x + 1` și `inc = \x -> x + 1` definesc exact aceeași funcție. Prima e doar o prescurtare.

`\x y -> …` are doi parametri; `\(x, y) -> …` are unul singur: o pereche.

Exemplu: flip în trei feluri

din Curs 3

```
flip :: (a -> b -> c) -> (b -> a -> c)

flip f x y = f y x              -- cu șabloane

flip f = \x y -> f y x          -- întoarce o lambda

flip = \f x y -> f y x          -- flip e o valoare
```

ghci

```
ghci> flip (-) 1 10
9
ghci> flip (:) [2,3] 1
[1,2,3]
ghci> (-) 1 10
-9
```

Funcție de ordin înalt = o funcție care primește o funcție ca argument sau întoarce o funcție ca rezultat. `flip` le face pe amândouă.

Currying: orice funcție are un singur argument

```
add :: Int -> (Int -> Int)
```

primește un Int

întoarce o funcție Int -> Int




```
add 3 4 = (add 3) 4
```

De aceea scriem `add 3 4`, nu `add(3, 4)`: nu există funcții „cu două argumente”, doar funcții care întorc funcții.

Două reguli

- **->** e asociativă la dreapta
- **aplicarea** e asociativă la stânga

Aplicare parțială în practică

ghci

```
ghci> add :: Int -> Int -> Int; add x y = x + y
ghci> :t add 3
add 3 :: Int -> Int
ghci> map (add 3) [1,2,3]
[4,5,6]
ghci> :t take 5
take 5 :: [a] -> [a]
ghci> map (take 2) ["haskell", "curry"]
["ha","cu"]
```

Cu sau fără lambda

```
map (\x -> add 3 x) xs
map (add 3) xs   -- la fel
```

Dacă lambda doar trece argumentul mai departe, o poți scoate.

Aplicare parțială: dai mai puține argumente și primești înapoi o funcție care le așteaptă pe restul.

curry și uncurry

tipuri

```
curry   :: ((a, b) -> c) -> a -> b -> c
uncurry :: (a -> b -> c) -> (a, b) -> c
```

din Curs 3

```
f :: (Int, String) -> String
f (n, s) = take n s
```

```
ghci> cf = curry f
ghci> :t cf
cf :: Int -> String -> String
ghci> f (1, "abc")
"a"
ghci> cf 1 "abc"
"a"
```

uncurry cu zip

```
ghci> uncurry (+) (3, 4)
7
ghci> map (uncurry (+)) (zip [1,2] [10,20])
[11,22]
```



`f (1, "abc")` nu e un apel cu două argumente: f primește **o singură pereche**.

Secțiuni: operatori aplicați parțial

Secțiune	Înseamnă	Exemplu
`(*3)`	`\x -> x * 3`	`(*3) 5` → `15`
`(10-)`	`\x -> 10 - x`	`(10-) 3` → `7`
`(^2)`	`\x -> x ^ 2`	`(^2) 3` → `9`
`(2^)`	`\x -> 2 ^ x`	`(2^) 3` → `8`
`(`elem` [2,3])`	`\x -> elem x [2,3]`	`(`elem` [2,3]) 3` → `True`
`(1 `elem`)`	`\xs -> elem 1 xs`	`(1 `elem`) [2,3]` → `False`

```
ghci> map (subtract 1) [1,2,3]
[0,1,2]
```

Capcană

`(-3)` e numărul −3, nu o secțiune. Pentru „scade 3” folosește:

```
subtract 3
```

aplica2: patru definiții echivalente

din Laboratorul 4

```
aplica2 :: (a -> a) -> a -> a

aplica2 f x = f (f x)          -- 1. direct
aplica2 f   = f . f            -- 2. compunere
aplica2     = \f x -> f (f x) -- 3. lambda
aplica2 f   = \x -> f (f x)   -- 4. întoarce o lambda
```

ghci

```
ghci> aplica2 (*2) 5
20
ghci> aplica2 tail [1..5]
[3,4,5]
ghci> aplica2 (++ "!") "hey"
"hey!!"
```



Întrebare: cât face `aplica2 (aplica2 (*2)) 1`? Hint: funcția interioară dublează de două ori.

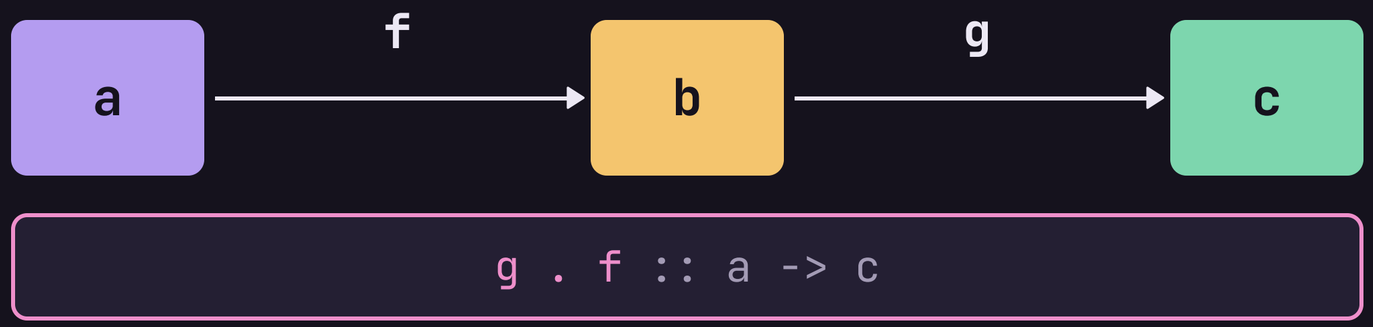
02

Compunere

Operatorul `.` (compunere) și operatorul `$` (aplicare cu precedență mică) (Curs 3).



Compunerea: operatorul (.)



definiția

```
(.) :: (b -> c) -> (a -> b) -> (a -> c)
(g . f) x = g (f x)
```

din Curs 3

```
ghci> :t take 5 . reverse
take 5 . reverse :: [a] -> [a]
ghci> (take 5 . reverse) [1..10]
[10,9,8,7,6]
ghci> (head . reverse . take 5) [1..10]
5
```

Cum citești

`head . reverse . take 5`

1. întâi take 5
2. apoi reverse
3. apoi head

De la dreapta la stânga, ca în matematică.

Operatorul $: mai puține paranteze

definiția

```
($) :: (a -> b) -> a -> b
f $ x = f x          -- precedență 0, asociativ la dreapta
```

`head (reverse (take 5 [1..10]))`	paranteze imbricate	→ 5
`(head . reverse . take 5) [1..10]`	compunere + paranteze	→ 5
`head . reverse . take 5 $ [1..10]`	compunere + $	→ 5
`head $ reverse $ take 5 $ [1..10]`	doar $	→ 5

Regula: tot ce e la dreapta lui $ se calculează întâi, ca și cum ar fi între paranteze.

Capcane și trucuri cu . și $

Greșit

```
ghci> head . reverse [1,2,3]
error: Couldn't match expected type …
       -- reverse [1,2,3] e o listă, nu o funcție
```

Corect

```
ghci> (head . reverse) [1,2,3]
3
ghci> head . reverse $ [1,2,3]
3
```

⚠ Aplicarea (spațiul dintre funcție și argument) leagă **mai strâns** decât orice operator, inclusiv `.`

Truc: o listă de funcții

```
ghci> map ($ 3) [(4 +), (10 *), (^ 2), sqrt]
[7.0,30.0,9.0,1.7320508075688772]
```

`($ 3)` = „aplică funcția pe 3”

`map ($ x) [f1, …, fn]` = `[f1 x, …, fn x]`

Stil point-free

```
f xs = sum (map (^2) xs)
f    = sum . map (^2)    -- fără xs
```

03

map și filter

Transformarea fiecărui element și selectarea elementelor care respectă o condiție (Curs 3, Laborator 4).



Trei unelte pentru liste



aceeași lungime, fiecare element transformat



unele elemente rămân, altele dispar



toată lista devine o singură valoare

Toate trei primesc o **funcție** ca argument: sunt funcții de ordin înalt.

De la recursie la map

pătrate (recursiv)

```
squares :: [Int] -> [Int]
squares []     = []
squares (x:xs) = x * x : squares xs
```

coduri ASCII (recursiv)

```
ords :: [Char] -> [Int]
ords []     = []
ords (x:xs) = ord x : ords xs
```

```
ghci> squares [1,-2,3]
[1,4,9]
ghci> ords "a2c3"
[97,50,99,51]
```


abstractizare: map

```
map :: (a -> b) -> [a] -> [b]
map f []     = []
map f (x:xs) = f x : map f xs

-- sau: map f xs = [f x | x <- xs]
```

cu map

```
squares xs = map sqr xs
  where sqr x = x * x
ords xs    = map ord xs
```

map: exemple de citit

`map (\x -> 2 * x) [1..5]`	→ `[2,4,6,8,10]`	lambda
`map (1 `elem`) [[2,3], [1,2]]`	→ `[False,True]`	e 1 în fiecare listă?
`map (`elem` [2,3]) [1,3,4,5]`	→ `[False,True,False,False]`	e fiecare x în [2,3]?
`map (* 3) [1,3,4]`	→ `[3,9,12]`	secțiune
`map length ["ab", "", "xyz"]`	→ `[2,0,3]`	funcție cu nume
`map (take 1) ["ab", "cd"]`	→ `["a","c"]`	aplicare parțială

De la recursie la filter

elemente pozitive (recursiv)

```
positives :: [Int] -> [Int]
positives [] = []
positives (x:xs)
  | x > 0     = x : positives xs
  | otherwise = positives xs
```

cu filter

```
positives = filter (> 0)
digits    = filter isDigit
```

Predicat = funcție care întoarce Bool: `(> 0)`, `isDigit`, `even`.

abstractizare: filter

```
filter :: (a -> Bool) -> [a] -> [a]
filter p [] = []
filter p (x:xs)
  | p x       = x : filter p xs
  | otherwise = filter p xs

-- sau: filter p xs = [x | x <- xs, p x]
```

```
ghci> positives [1,-2,3]
[1,3]
ghci> digits "a2c3"
"23"
ghci> filter odd [3,1,4,2,5]
[3,1,5]
```

map + filter = descriere de listă

```
[f x | x <- xs, p x]   ==   map f (filter p xs)
```

map f

filter p

aceeași listă

```
ghci> [x * x | x <- [1..6], odd x]
[1,9,25]
ghci> map (\x -> x * x) (filter odd [1..6])
[1,9,25]
```

ordinea contează

```
ghci> map (+1) (filter even [1..6])
[3,5,7]
ghci> filter even (map (+1) [1..6])
[2,4,6]
```



`map f . filter p` și `filter p . map f` nu sunt același lucru: filtrul vede valori diferite.

04

fold

Agregarea unei liste într-o singură valoare: foldr, foldl și diferența dintre ele (Cursurile 3–4, Laboratorul 5).



Același tipar, a treia oară

```haskell
sum :: [Int] -> Int
sum []     = 0
sum (x:xs) = x + sum xs
```

```haskell
product :: [Int] -> Int
product []     = 1
product (x:xs) = x * product xs
```

```haskell
concat :: [[a]] -> [a]
concat []     = []
concat (y:ys) = y ++ concat ys
```

operația: `+`

lista vidă: `0`

operația: `*`

lista vidă: `1`

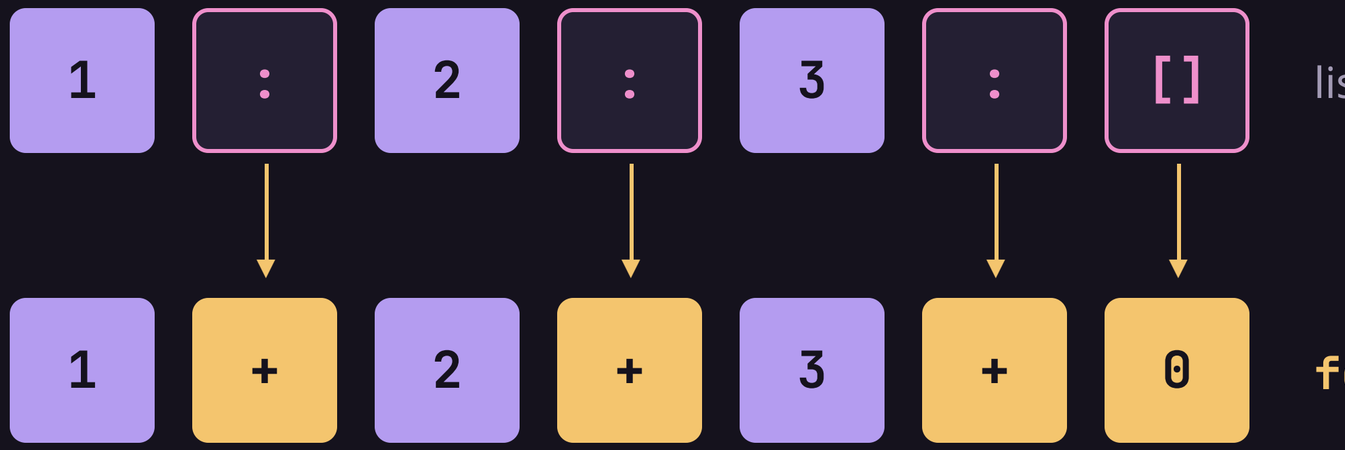
operația: `++`

lista vidă: `[]`

abstractizare: foldr

```haskell
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f i []     = i
foldr f i (x:xs) = f x (foldr f i xs)
```

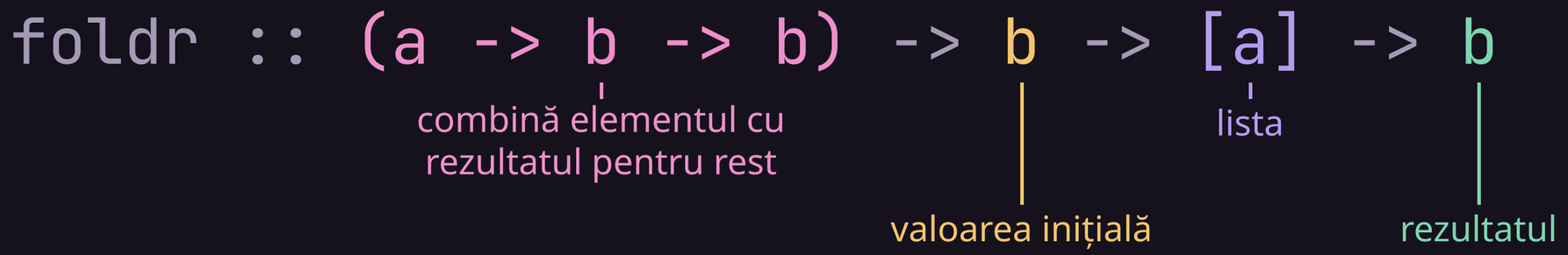
foldr înlocuiește constructorii




```
foldr (+) 0 [1,2,3]  ==  1 + (2 + (3 + 0))  ==  6
```

Σ **Regula:** fiecare `:` devine `f`, lista vidă `[]` devine valoarea inițială. Parantezele se grupează la dreapta.

Cum citești tipul lui foldr



din Curs 3

```
sum     = foldr (+) 0          -- 1 + (2 + (3 + 0))
product = foldr (*) 1          -- 1 * (2 * (3 * 1))
concat  = foldr (++) []        -- "Hello" ++ ("world" ++ ("!" ++ []))
```

a și b pot fi diferite: numără elementele pare

```
ghci> foldr (\x acc -> if even x then acc + 1 else acc) 0 [1,2,3,4]
2
```

Suma pătratelor pozitivelor: patru variante

1 · where

```
f xs = foldr (+) 0 (map sqr (filter pos xs))
  where sqr x = x * x
        pos x = x > 0
```

2 · lambda

```
f xs = foldr (+) 0
         (map (\x -> x * x)
           (filter (\x -> x > 0) xs))
```

3 · secțiuni

```
f xs = foldr (+) 0 (map (^ 2) (filter (> 0) xs))
```

4 · point-free

```
f = foldr (+) 0 . map (^2) . filter (>0)
```

```
ghci> f [1,-2,3]
10
```

Toate variantele au tipul `f :: [Int] -> Int` și dau același rezultat. Recunoaște-le pe toate.

Fold poate întoarce și funcții

din Curs 4

```
compose :: [a -> a] -> (a -> a)
compose = foldr (.) id

id :: a -> a
id x = x
```

```
ghci> compose [(+1), (^2)] 3
10
ghci> compose [(*2), (+1), (*10)] 1
22
```

```
foldr (.) id [f, g, h]  ==  f . (g . (h . id))  ==  f . g . h
```



Element neutru: `0` pentru `+`, `1` pentru `*`, `[]` pentru `++`, `id` pentru `.` El e mereu valoarea inițială a lui fold.

foldl: de la stânga la dreapta

foldr

```
foldr :: (a -> b -> b) -> b -> [a] -> b
```

```
a1 `op` (a2 `op` (… (an `op` z)))
```

funcția primește **(element, rest)**

grupare la dreapta

foldl

```
foldl :: (b -> a -> b) -> b -> [a] -> b
```

```
((z `op` a1) `op` a2) … `op` an
```

funcția primește **(acumulator, element)**

grupare la stânga

definiția

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl h i []     = i
foldl h i (x:xs) = foldl h (h i x) xs
```

suml din Curs 4 = foldl (+) 0

```
sum xs = suml xs 0
  where suml []     n = n
        suml (x:xs) n = suml xs (n + x)
```

Când contează direcția

operație neasociativă

```
ghci> foldr (-) 0 [1,2,3]      -- 1 - (2 - (3 - 0))
2
ghci> foldl (-) 0 [1,2,3]      -- ((0 - 1) - 2) - 3
-6
```

din Laboratorul 5

```
ghci> foldl (++) "first" ["abc","def","ghi"]
"firstabcdefghi"
ghci> foldr (++) "last" ["abc","def","ghi"]
"abcdefghilast"
```

Pe scurt

- **+, *, ++, &&, ||:** același rezultat
- **-, /, construcții de liste:** rezultate diferite
- **valoarea inițială** stă la capătul dinspre care pornește fold-ul



Nu știi ce dă? **Scrie parantezele de mână** pentru o listă de trei elemente. Merge mereu.

Inversarea unei liste cu foldl

din Curs 4

```
rev :: [a] -> [a]
rev = foldl (flip (:)) []

-- (:)       :: a -> [a] -> [a]
-- flip (:) :: [a] -> a -> [a]
```

rev [1,2,3], pas cu pas

start	acc = `[]`	
x = 1	acc = `[1]`	`flip (:) [] 1`
x = 2	acc = `[2,1]`	`flip (:) [1] 2`
x = 3	acc = `[3,2,1]`	`flip (:) [2,1] 3`



Varianta cu foldr: `foldr (\x acc -> acc ++ [x]) []`. Merge, dar e mai lentă: fiecare `++` parcurge din nou acumulatorul.

05

Lazy, din nou

map, filter și fold pe liste infinite: de ce foldr poate, iar foldl nu (Curs 4).



map și filter pe liste infinite

din Curs 4

```
ghci> inf = map (+10) [1..]      -- nu se calculează nimic
ghci> take 3 inf
[11,12,13]
ghci> take 2 (filter even (map (^2) [1..]))
[4,16]
ghci> take 3 (map (*2) (filter odd [1..]))
[2,6,10]
```

1 **când**
doar când e nevoie de valoare

2 **cât**
doar elementele cerute

3 **o dată**
o valoare calculată se refolosește

∞ Lista infinită nu e o problemă cât timp **cineva oprește cererea** (take, head, !!). Fără oprire, nici map, nici filter nu se termină.

foldr poate, foldl niciodată

foldr: se termină

```
ghci> take 3 $ foldr (\x xs -> (x+1):xs) [] [1..]
[2,3,4]
ghci> foldr (\x acc -> x > 10 || acc) False [1..]
True
```

nu se termină

```
ghci> take 3 $ foldl (\xs x -> (x+1):xs) [] [1..]
-- nu se termină niciodată (Ctrl+C)
ghci> foldr (*) 0 [1..]
-- nici asta: * are nevoie de ambele argumente
```

De ce?

- **foldl** trebuie să ajungă la capătul listei înainte să întoarcă ceva.
- **foldr** poate întoarce un rezultat parțial dacă funcția nu are nevoie de „rest”.

La examen: „care se termină?” Verifică dacă funcția dată lui foldr poate răspunde fără al doilea argument.

06

Exerciții

Încălzire cu descrieri de liste, apoi opt probleme din Laboratoarele 4 și 5. Soluțiile urmează.



Încălzire: ce valoare au?

1
```
[x^2 | x <- [1..10], x `rem` 3 == 2]
```

2
```
[(x,y) | x <- [1..5], y <- [x..(x+2)]]
```

3
```
[(x,y) | x <- [1..3], let k = x^2, y <- [1..k]]
```

4
```
[x | x <- "Facultatea de Matematica si Informatica", elem x ['A'..'Z']]
```

5
```
[[x..y] | x <- [1..5], y <- [1..5], x < y]
```

6
```
map (`elem` [2,3]) [1,3,4,5]
```

Încălzire: răspunsuri

1	`[4,25,64]`	$x \in \{2, 5, 8\}$: restul împărțirii la 3 este 2
2	`[(1,1),(1,2),(1,3),(2,2),(2,3),(2,4),(3,3),…,(5,7)]`	15 perechi: y merge de la x la x + 2
3	`[(1,1),(2,1),(2,2),(2,3),(2,4),(3,1),…,(3,9)]`	14 perechi: y merge de la 1 la x^2
4	`"FMI"`	doar majusculele
5	`[[1,2],[1,2,3],[1,2,3,4],[1,2,3,4,5],[2,3],…,[4,5]]`	10 intervale
6	`[False,True,False,False]`	e x în [2,3]?

Setul de exerciții

1 L4·2–4

`factori, prim`

Divizorii lui n, test de primalitate, numerePrime 20 → [2,3,5,7,11,13,17,19]

2 L4·8

`prel2`

Pare înjumătățite, impare dublate. [2,4,5,6] → [1,2,10,3]

3 L4·11

`pătrate pe poziții impare`

Pătratele elementelor de pe pozițiile impare. Hint: zip

4 L4·12

`numaiVocale`

["laboratorul", "PrgrAmare"] → ["aoaou", "Aae"]

5 L5·3–4

`allVerifies, anyVerifies`

Toate / măcar un element respectă o proprietate. Cu foldr.

6 L5·5

`mapFoldr, filterFoldr`

Rescrie map și filter folosind doar foldr.

7 L5·6

`listToInt`

Cu foldl: [2,3,4,5] → 2345

8 L5·7

`rmChar, rmCharsFold`

rmCharsFold ['a'..'l'] "halloween" → "own"

Soluții 1–2

1 · factori, prim, numerePrime

```haskell
factori :: Int -> [Int]
factori n = [d | d <- [1..n], n `mod` d == 0]

prim :: Int -> Bool
prim n = factori n == [1, n]

numerePrime :: Int -> [Int]
numerePrime n = [x | x <- [2..n], prim x]
```



`prim 1` dă False: `factori 1 = [1]`, care nu e egal cu `[1, 1]`.

2 · prel2

```haskell
prel2 :: [Int] -> [Int]
prel2 = map (\x -> if even x
                   then x `div` 2
                   else x * 2)
```

verificare

```
ghci> numerePrime 20
[2,3,5,7,11,13,17,19]
ghci> prel2 [2,4,5,6]
[1,2,10,3]
```

Soluții 3–4

3 · pătrate pe poziții impare

```
patratePozImpare :: [Int] -> [Int]
patratePozImpare xs =
  map ((^2) . snd) (filter (odd . fst) (zip [0..] xs))
```

4 · numaiVocale

```
numaiVocale :: [String] -> [String]
numaiVocale = map (filter (`elem` "aeiouAEIOU"))
```

verificare

```
ghci> patratePozImpare [10,1,20,2,30,3]
[1,4,9]
ghci> numaiVocale ["laboratorul", "PrgrAmare", "DEclarativa"]
["aoaou","Aae","Eaaia"]
```

Soluții 5–6: totul cu foldr

5 · allVerifies, anyVerifies

```
allVerifies :: (Int -> Bool) -> [Int] -> Bool
allVerifies p = foldr (\x acc -> p x && acc) True

anyVerifies :: (Int -> Bool) -> [Int] -> Bool
anyVerifies p = foldr (\x acc -> p x || acc) False

toateTrue :: [Bool] -> Bool
toateTrue = foldr (&&) True
```

6 · mapFoldr, filterFoldr

```
mapFoldr :: (a -> b) -> [a] -> [b]
mapFoldr f = foldr (\x acc -> f x : acc) []

filterFoldr :: (a -> Bool) -> [a] -> [a]
filterFoldr p = foldr (\x acc -> if p x
                                 then x : acc
                                 else acc) []
```

verificare

```
ghci> allVerifies even [2,4,6]
True
ghci> anyVerifies (> 5) [1,2,3]
False
ghci> filterFoldr odd [1..6]
[1,3,5]
```

Σ

Valoarea inițială = **elementul neutru**: `True` pentru `&&`, `False` pentru `||`, `[]` pentru liste.

Soluții 7–8: foldl și foldr cu funcții existente

7 · listToInt

```
listToInt :: [Integer] -> Integer
listToInt = foldl (\acc d -> acc * 10 + d) 0
```

8 · rmChar, rmCharsRec, rmCharsFold

```
rmChar :: Char -> String -> String
rmChar c = filter (/= c)

rmCharsRec :: String -> String -> String
rmCharsRec []     s = s
rmCharsRec (c:cs) s = rmCharsRec cs (rmChar c s)

rmCharsFold :: String -> String -> String
rmCharsFold cs s = foldr rmChar s cs
```

verificare

```
ghci> listToInt [2,3,4,5]
2345
ghci> rmCharsFold ['a'..'l'] "halloween"
"own"
```

`0 → 2 → 23 → 234 → 2345`

Fiecare cifră: acc * 10 + d.

`rmChar :: Char -> String -> String` are exact forma `a -> b -> b` cerută de foldr, deci o dai direct, fără lambda.

07

Recapitulare

Capcanele frecvente, fițuica prezentării și quiz-ul de final.



Capcane frecvente

`(-3)`

e numărul −3, nu o secțiune; folosește (subtract 3)

`head . reverse [1,2,3]`

lipsește $ sau paranteze: aplicarea leagă mai strâns

`foldl pe [1..]`

nu se termină niciodată

`\acc x vs \x acc`

foldl: (acc, x); foldr: (x, acc)

`foldr (-) vs foldl (-)`

pentru operații neasociative rezultatele diferă

`map f . filter p`

nu e același lucru cu filter p . map f

`map (+1) 5`

map vrea o listă; pe o valoare scrii (+1) 5

`\x y -> vs \(x, y) ->`

doi parametri vs. o singură pereche

Fițuica prezentării 2

Lambda, currying

```
\x -> x + 1
\x y -> x + y

f :: a -> b -> c
  = a -> (b -> c)
f x y = (f x) y

flip f x y = f y x
curry, uncurry

(*3)  (10-)  (^2)
(`elem` xs)
subtract 3
```

Compunere

```
(.) :: (b -> c)
    -> (a -> b)
    -> a -> c
(g . f) x = g (f x)

($) :: (a -> b)
    -> a -> b
f $ x = f x

f . g . h $ x
map ($ 3) fs
```

map, filter

```
map :: (a -> b)
    -> [a] -> [b]

filter :: (a -> Bool)
       -> [a] -> [a]

[f x | x <- xs, p x]
 == map f
      (filter p xs)
```

fold

```
foldr :: (a -> b -> b)
      -> b -> [a] -> b
foldl :: (b -> a -> b)
      -> b -> [a] -> b

foldr: x1 op (… z)
foldl: (z op x1) …

sum = foldr (+) 0
rev = foldl
  (flip (:)) []
```

Quiz de final

1. Ce tip are `map (*2)`?

2. Ce întoarce `foldr (\x acc -> acc ++ [x]) [] [1,2,3]`?

3. Ce întoarce `(filter even . map (+1)) [1..5]`?

4. Cât face `foldl (-) 10 [1,2,3]`?

5. Ce întoarce `take 3 (map (*2) (filter odd [1..]))`?

Quiz de final: răspunsuri

1	`Num b => [b] -> [b]`	map cu aplicare parțială: așteaptă lista
2	`[3,2,1]`	elementul curent merge la sfârșit: inversare
3	`[2,4,6]`	întâi +1: [2..6], apoi păstrăm parele
4	`4`	`((10 - 1) - 2) - 3`
5	`[2,6,10]`	lazy: din [1..] se iau doar 1, 3, 5

URMEAZĂ · PREZENTAREA 3

Tipuri de date algebrice

Tipuri sumă și produs, Maybe, Either, înregistrări, arbori (Curs 5, Laborator 6–7)

Până data viitoare

Termină Lab 4 și Lab 5: myzip3, mymap, myfilter, myReverse, myElem, myUnzip, union, intersect, plus X și 0 de la Extra. Citește din Learn You a Haskell capitolul Higher Order Functions.